

IAGENTE

infraestrutura de comunicação multicanal

Avaliação Técnica

Desenvolvedor(a) PHP · Laravel — Full-Stack

Nível Pleno

Critérios da avaliação

Nesta avaliação propomos um exercício de programação no qual você poderá demonstrar seu conhecimento em desenvolvimento de software full-stack.

Valorizamos código de boa qualidade: simples, fácil de ler, entender e alterar. Avaliamos a **simplicidade, clareza e organização** do código — o quão “limpo” ele está — além de **decisões de arquitetura, tratamento de erros, componentização e testes**. Não buscamos a solução mais complexa, e sim a mais correta e sustentável.

O exercício deve ser feito inteiramente com **Laravel (versão 11 ou mais atual)** sobre **PHP 8.2+**, construindo a interface com **Blade + Livewire** (Inertia/TailWind também é aceito). O produto final é uma **aplicação web funcional** — front e back no mesmo projeto. **Não é necessário (nem esperado) React ou Next.js**.

Inclua um arquivo **README.md** com instruções de execução e as decisões que você tomou ao longo do desenvolvimento.

O resultado deve ser enviado conforme as instruções ao final deste documento.

Descrição do projeto

Desenvolva uma **aplicação web full-stack em Laravel** que consuma a **PokeAPI (<https://pokeapi.co>)** e permita ao usuário buscar Pokémons, ver detalhes, favoritar e conversar em tempo real com outros usuários.

A avaliação olha para os dois lados: no **frontend**, como você estrutura os componentes Livewire, cuida do estado, da reatividade e de uma UX simples e coerente; no **backend**, como modela os dados (Eloquent), protege as rotas, e consome a PokeAPI de forma **resiliente e eficiente** (cache, tratamento de indisponibilidade, sem sobrecarregar o serviço externo) — o mesmo tipo de desafio que enfrentamos integrando gateways de e-mail, SMS e WhatsApp.

Funcionalidades requeridas

1. Autenticação

- Telas de login e logout; a aplicação (exceto login/cadastro) só é acessível autenticado.
- Pode usar um starter kit (**Laravel Breeze** com stack Livewire, ou Fortify) — não precisa reinventar a autenticação.
- Cada usuário enxerga apenas os seus próprios favoritos e seu próprio perfil.

2. Cadastro de usuário

- Tela pública de registro (nome, e-mail, senha) com validação e feedback de erro na interface.
- Senha armazenada com hash (nunca em texto puro); validação via **Form Request** ou regras no componente.

3. Chat em tempo real (WebSockets / Broadcasting)

- Interface de chat em tempo real com **Livewire + broadcasting do Laravel** (Laravel Reverb, Pusher ou similar via Laravel Echo).
- O usuário deve listar outros usuários da plataforma e trocar mensagens diretas, que aparecem sem recarregar a página.

- Mensagens **persistidas** (histórico recuperável) e entregues via **Event + Broadcast** em canal privado/presence.

4. Busca de Pokémon

- Campo de busca reativa por nome ou por tipo (água, fogo, etc.) — busca ao digitar (`wire:model.live` com `debounce`) e/ou por botão.
- Dados vindos da PokeAPI. **Obrigatório implementar cache** das respostas para não chamar a API externa a cada tecla/requisição.
- Tratar Pokémon inexistente e indisponibilidade da PokeAPI com mensagens claras na interface.

5. Lista de resultados

- Exibir os resultados em cards/lista, cada item com nome e imagem, no mínimo — e outros dados que julgar relevantes.

6. Detalhes do Pokémon

- Ao clicar num item, abrir uma página/tela de detalhes com nome, imagem, tipo(s), habilidades e demais informações relevantes.
- Reaproveitar o cache da busca quando fizer sentido.

7. Navegação

- Navegação entre as páginas: Início (busca), Favoritos, Detalhes do Pokémon e Meu Perfil, com um layout consistente.

8. Paginação

- Paginação nos resultados (ex.: `trait WithPagination` do Livewire), preparada para lidar com listas grandes.

9. Favoritos

- Adicionar, listar e remover Pokémon dos favoritos direto pela interface; página dedicada de favoritos (rota ou modal, à sua escolha).
- Persistência em banco com **relacionamento Eloquent** (ex.: usuário N:N pokémons favoritos).
- Garantir **idempotência**: favoritar o mesmo Pokémon duas vezes não duplica registro.

10. Meu perfil

- Tela onde o usuário visualiza e altera os próprios dados (nome e senha).
- Alteração de senha com validação e novo hash; um usuário nunca altera dados de outro.

Requisitos não funcionais

- Instruções claras de como rodar e acessar a aplicação, incluindo **credenciais semeadas de ADMIN e USER** (via seeder).
- Fornecer `.env.example`, migrations e seeders. Deve subir com `migrate --seed` sem passos manuais escondidos.
- Hospedar em plataforma como **Railway**, **Render**, **Fly.io** ou similar; ou fornecer **docker-compose** (app + MySQL + Redis) que suba com um comando.

Requisitos técnicos

- **Laravel 11+** sobre **PHP 8.2+**.
- Interface com **Blade + Livewire** (preferencial) ou **Inertia**. Estilização livre — usamos **Tailwind**.
- **MySQL** (ou PostgreSQL) como banco. SQLite só se justificado no README.
- Camada de **cache** para a PokeAPI (Redis preferível; file/database aceitos).
- Pelo menos alguns **testes automatizados** (Pest ou PHPUnit) nos caminhos críticos — autenticação, favoritos e busca com cache.
- PokeAPI: <https://pokeapi.co>

Diferenciais (nos surpreenda)

Não são obrigatórios, mas contam pontos e ajudam a te posicionar como pleno/sênior:

- Uso de **filas (queues/jobs)** — ex.: cachear dados da PokeAPI em background.
- **Rate limiting** e **throttle** nas chamadas à PokeAPI; **timeout + retry com backoff** no client `Http::`.
- Separação de responsabilidades (**Service/Action, Form Requests, Policies**), componentes Livewire pequenos e reutilizáveis.
- Testes de feature cobrindo autorização — garantir que um usuário não acessa dados de outro (**IDOR**).
- Atenção a UX: estados de loading, mensagens de erro amigáveis, responsividade básica.

Sugestões

- Se não conseguir finalizar todos os requisitos, priorize entregar os mais viáveis por completo — é melhor do que entregar todos pela metade.
- Se sobrar tempo, fique à vontade para propor melhorias e novas funcionalidades.
- O **Laravel Reverb** roda localmente e integra direto com Livewire/Echo — costuma ser o caminho mais rápido para o chat. Pusher também é aceito.
- Comentários no código que expliquem decisões não óbvias são bem-vindos.

Stack que usamos hoje

Você é livre para usar qualquer biblioteca ou ferramenta. Segue o que usamos no dia a dia, caso queira se familiarizar:

Ferramenta	Uso
Laravel + Blade	Base da aplicação e views
Livewire	Componentes reativos full-stack (interface)
Alpine.js	Interatividade leve no cliente
Tailwind CSS	Estilização
Laravel Breeze / Fortify	Autenticação e cadastro
Laravel Reverb	WebSockets / broadcasting em tempo real
Eloquent ORM	Modelagem e relacionamentos
Redis	Cache e filas
Laravel Horizon	Observabilidade de filas

Ferramenta	Uso
Pest / PHPUnit	Testes automatizados
MySQL	Banco de dados relacional

Se optar por **Inertia**, no front usamos Tailwind, Shadcn e Tanstack Query — mas **Livewire é o caminho recomendado** para esta vaga.

Instruções para envio

Crie um **repositório privado no GitHub** com o resultado do exercício:

- Adicione como colaborador o usuário: [tittobreno](#)
- Registre no **README.md**: passo a passo de execução, credenciais de ADMIN e USER, decisões técnicas, o que ficou pronto e o que ficou de fora (e por quê).
- Se hospedar a aplicação, inclua a URL e as credenciais de acesso no README.

Boa sorte — estamos animados para ver como você resolve.